GUDSKY RESEARCH FOUNDATION

Virtual Research Center (VRC) · Dronacharya Free Learning

AI Model Creation & Deployment

5-Day Hands-On Workshop

25 March 2026 – 29 March 2026 · Online · 100% Free of Cost

ObjectVision

Real-Time Object Detection System

Official Project Documentation

Prepared for Student Reference and Technical Study

Technology Stack

YOLOv8 · FastAPI · WebSocket · OpenCV · Python · SQLite

ISO Certified | MSME Certified | AICTE Approved | Startup India · DPIIT | NGO Darpan

Chapter 1 — Introduction and Project Overview 4
1.1 What is ObjectVision? 4
1.2 Core Capabilities 4
1.3 Technology Stack Overview 5
Chapter 2 — System Architecture 7
2.1 The Layered Architecture 7
2.2 How a Request Flows Through the System 7
2.3 Module Reference 8
2.4 Project File Structure 9
Chapter 3 — Installation and Quick Start Guide 11
3.1 Prerequisites ..... **Error! Bookmark not defined.**
3.2 Installation Steps 11
Step 1 — Obtain the Project Files 11
Step 2 — Run the Setup Script 11
Step 3 — Configure Your Environment 12
Step 4 — Activate the Virtual Environment and Start the Server 12
Step 5 — Open the Application 13
Chapter 4 — API Reference 14
4.1 Complete Endpoint Reference 14
4.2 Understanding HTTP Methods 15
4.3 Rate Limiting 15
Chapter 5 — The WebSocket Inference Protocol 17
5.1 What is a WebSocket and Why Use It? 17
5.2 Connection Lifecycle and Message Sequence 17
Phase 1 — Authentication and Connection 17
Phase 2 — Configuration Frame 17
Phase 3 — Binary Video Upload 18
Phase 4 — Streaming Frame Results 18
5.3 Server Status and Error Messages 18
Chapter 6 — YOLOv8 Model Reference 20
6.1 What is YOLO? 20
6.2 Model Variants and the Speed-Accuracy Trade-off 20
6.3 Hot-Swapping Models at Runtime 21
6.4 The COCO 80 Object Classes 21
Chapter 7 — Authentication and API Key Management 23
7.1 Understanding API Keys 23
7.2 Secure Storage with SHA-256 Hashing 23
7.3 Key Management via Command Line 23

7.4 Key Management via the Admin API 24
7.5 Disabling Authentication for Development 24
Chapter 8 — Environment Configuration Reference 25
Chapter 9 — Video Storage and Logging 27
9.1 Video Storage Organisation 27
9.2 File Naming Convention 27
9.3 Supported Video Formats 27
9.4 Understanding the Log Files 28
The Access Log (logs/access.log) 28
The Inference Log (logs/inference.log) 28
Viewing Logs via the Admin API 28
Chapter 10 — Attribution and Acknowledgements 30
10.1 Gudsky Research Foundation 30
10.2 Virtual Research Center 30
10.3 Dronacharya Free Learning Initiative 30
10.4 The Workshop 30

Chapter 1 — Introduction and Project Overview

Welcome to the official documentation of ObjectVision, a real-time object detection system developed at the Virtual Research Center of the Gudsky Research Foundation. This document is intended to serve as a complete technical reference for students and researchers participating in the AI Model Creation and Deployment Workshop. Whether you are new to artificial intelligence, computer vision, or web development, this guide is written in a way that builds your understanding step by step.

Object detection is one of the most exciting and practical areas of Artificial Intelligence. At its core, it answers a very simple question: given an image or a video frame, what objects are present, and where exactly are they? This ability is used everywhere around us — in self-driving cars that recognise pedestrians and traffic lights, in hospital systems that scan medical images for tumours, in retail stores that track inventory on shelves, and in security cameras that detect unusual activity.

ObjectVision brings this technology directly to your browser. You upload a video, the system processes it frame by frame, identifies every object it recognises, draws coloured bounding boxes around them with confidence percentages, and streams the results back to your screen in real time. The entire pipeline — from the moment you upload the video to the moment you see the annotated output — is built using modern, industry-standard tools that you will encounter throughout your career in AI and software development.

1.1 What is ObjectVision?

ObjectVision is a full-stack, production-style web application that performs real-time object detection on video files. It is built using a layered architecture: a browser-based user interface for interaction, a FastAPI server for handling requests, a WebSocket channel for streaming data in real time, and a YOLOv8 neural network model for the actual object detection.

The word 'full-stack' means that ObjectVision covers both the front end (what the user sees and interacts with) and the back end (the server logic, database, and AI model). This makes it a complete, deployable application, not just a standalone Python script. By studying ObjectVision, you will learn how modern AI projects are structured in the real world.

> **Why is this important for students?**
> Most AI tutorials teach you how to train a model or run inference in a Jupyter notebook. ObjectVision goes further — it shows you how to integrate that model into a real application with authentication, rate limiting, logging, and a live browser UI. This is the gap between academic AI and production AI.

1.2 Core Capabilities

The system is designed around seven major capabilities. Each one adds a layer of real-world readiness to the project:

- **Real-Time Inference via WebSocket:** Videos are not processed in bulk and returned after a long wait. Instead, the server processes each frame and immediately streams the annotated result back to the browser. The user sees a live, frame-by-frame playback of the detection results, much like watching a video with annotations overlaid on top.
- **Six YOLOv8 Model Variants:** From the tiny Nano model that runs efficiently on a laptop CPU, to the massive World model that achieves top accuracy on powerful GPUs. Students can switch between models and directly observe the trade-off between speed and accuracy.
- **Eighty COCO Object Classes:** The system can recognise eighty categories of everyday objects — people, vehicles, animals, furniture, food, electronics, and more. These classes come from the COCO (Common Objects in Context) dataset, which is the standard benchmark in computer vision research.

- **API Key Authentication:** In a real deployment, you cannot allow anyone on the internet to use your AI server without limits. ObjectVision implements an API key system where every user receives a unique key. Keys are stored as SHA-256 hashes in a SQLite database, meaning the plaintext key is never saved anywhere.
- **Per-Key Rate Limiting:** Even authenticated users are limited in how many requests they can make. This prevents one user from monopolising the server resources and ensures fair access for all.
- **Organised Video Storage:** Every video that is uploaded is automatically saved to a folder named after today's date. Each file also receives a unique suffix to prevent naming conflicts. This makes it easy to manage and retrieve stored videos.
- **Structured Logging:** Every request and every inference session is recorded to log files in JSON format. Administrators can tail these logs through a protected API endpoint to monitor usage and debug issues.

1.3 Technology Stack Overview

Each technology in the stack was chosen deliberately. The table below explains what each tool is, what role it plays in the project, and why it was chosen over alternatives.

Technology	Role in ObjectVision	Why This Tool?
YOLOv8	Object Detection	YOLO stands for 'You Only Look Once'. YOLOv8 is the latest generation, made by Ultralytics. Unlike older detection systems that scan an image multiple times, YOLO processes the entire image in a single forward pass through the neural network, making it extremely fast — ideal for real-time video.
FastAPI	Web Server & API	FastAPI is a modern Python web framework known for its speed and automatic documentation generation. It natively supports async programming (asyncio), which is essential for handling multiple simultaneous WebSocket connections without blocking.
WebSocket	Real-Time Stream	Unlike a normal HTTP request which opens, gets a response, and closes, a WebSocket keeps a persistent, two-way connection open. This allows the server to continuously push annotated frames to the browser without the client needing to make repeated requests.
OpenCV	Frame Processing	OpenCV (Open Source Computer Vision Library) is the industry standard for image and video processing in Python. It is used here to read video frames from uploaded files, encode them to JPEG format for transmission, and draw the bounding boxes and labels on each frame.
SQLite	Key Storage	SQLite is a lightweight, serverless database engine. It stores the hashed API keys and their metadata. For a project of this scale, SQLite is an excellent choice: it requires no separate server process and the entire database lives in a single file (data/keys.db).

Uvicorn	ASGI Server	Uvicorn is an ASGI (Asynchronous Server Gateway Interface) server. It is the engine that runs the FastAPI application. ASGI allows Python web applications to handle thousands of concurrent connections efficiently, which is critical when streaming video frames to many users simultaneously.

Chapter 2 — System Architecture

Understanding the architecture of a software system means understanding how its different parts are organized, how they communicate with each other, and why they are structured the way they are. ObjectVision follows a layered architecture, where each layer has a clearly defined responsibility and communicates only with the layers directly above and below it. This design principle is called Separation of Concerns, and it is fundamental to building maintainable, scalable software.

2.1 The Layered Architecture

ObjectVision has four distinct layers: the Client Layer, the Server Layer, the Intelligence Layer, and the Persistence Layer. A request from the user flows downward through these layers, and the response travels back upward.

Client Layer — The Browser UI:

This is the top layer, the part the student or end user directly interacts with. It is a single HTML page (index.html) served by the FastAPI server at the /static/ path. The page contains a file upload control, a model selection dropdown, a confidence threshold slider, a class filter input, a live canvas element that displays the annotated video frames, and a detection log panel that lists every detected object. The browser communicates with the server in two ways: standard HTTP REST calls for non-streaming operations (uploading files, listing storage), and a WebSocket connection for the real-time inference stream.

Server Layer — FastAPI:

This is the middle layer, the brain of the application. The file src/server.py defines all endpoints. When a request arrives, it first passes through middleware — code that runs before the actual endpoint handler. There are two middleware components: the RequestLoggingMiddleware (which records every request to a log file) and the CORS middleware (which allows the browser to make requests to the server). After middleware, the request reaches the endpoint handler, which may call out to the authentication system, the rate limiter, or the inference pipeline.

Intelligence Layer — YOLOv8 + OpenCV:

This layer contains the src/model/object_model.py file, which wraps the YOLOv8 model in a clean, reusable class called ObjectDetector. The detector loads the model weights once at startup, selects the best available hardware device (GPU if available, otherwise CPU), and exposes a simple detect() method. When called with a video frame, this method runs the neural network, collects the detection results, draws annotated bounding boxes directly on the frame, and returns a structured list of detected objects.

Persistence Layer — Storage and Databases:

This bottom layer handles everything that needs to be saved permanently. It consists of three parts: the Video_storage directory (organised by date), the data/keys.db SQLite database (holding hashed API keys), and the logs directory (containing access.log and inference.log in JSON-per-line format).

2.2 How a Request Flows Through the System

Let us trace the complete journey of a student who uploads a video for object detection. Understanding this flow will help you understand every component of the system.

1. The student opens the browser UI at http://localhost:8000/static/index.html. They select a video file, choose a YOLOv8 model variant, set a confidence threshold (for example, 0.5 means the model only reports detections it is at least 50% confident about), and optionally filter which classes to detect.

2. The browser opens a WebSocket connection to /ws/inference?api_key=ov_... The server checks the API key before even accepting the connection. If the key is invalid, it sends an error message and closes the connection immediately with code 4001 (Unauthorized).
3. If authentication passes, the rate limiter runs a check. If this user's key has exceeded its allowed number of connections, the server closes the connection with code 4029 (Too Many Requests).
4. The ConnectionManager registers the session (up to 50 simultaneous users are allowed), assigns a unique session ID, and marks the session status as 'connected'.
5. The browser sends a JSON text frame containing the configuration: the chosen model name, confidence threshold, class filter list, and the original filename.
6. If the requested model is different from the one currently loaded, the server hot-swaps the model weights. The student sees a status message: 'Loading yolov8m.pt…' followed by 'yolov8m.pt ready'.
7. The browser sends the video file as a binary WebSocket frame (raw bytes). The server saves these bytes to Video_storage/YYYY-MM-DD/ with a UUID suffix, then sends a status message confirming the save path.
8. OpenCV opens the saved video file, reads it frame by frame. For each frame, the ObjectDetector runs YOLOv8 inference, draws bounding boxes and labels in distinct colours, encodes the frame to JPEG at 75% quality, then base64-encodes it.
9. The server sends a JSON text frame back to the browser for each video frame, containing the frame index, current FPS, progress percentage, the base64 JPEG image, and the list of detections.
10. When all frames are processed, the server sends a final 'done' event with the total frame count and saved path. It then logs the full session details to inference.log.

2.3 Module Reference

The table below describes every source file in the project, explaining its purpose in detail so that students understand what to read when investigating a particular feature.

File	Detailed Purpose
app.py	The entry point of the entire application. When you run 'python app.py', this file is executed first. It loads the .env configuration file using the python-dotenv library, then imports the FastAPI app object from src/server.py, and finally starts the Uvicorn server. The reason configuration is loaded before the import is important: all the os.getenv() calls inside server.py happen at import time, so the environment variables must already be set before that file is read.
src/server.py	The heart of the application. This file defines the FastAPI application object, registers middleware, mounts the static file directory, creates the ObjectDetector and ConnectionManager singletons, and defines every REST and WebSocket endpoint. It is the central coordination point through which all requests flow.
src/websocket_handler.py	Defines the ConnectionManager class, which is responsible for tracking all active WebSocket sessions. It maintains a dictionary of Session objects, each holding the WebSocket connection, a session ID, the connection timestamp, a frame counter, and a status string. The class enforces the 50-user capacity limit, provides methods for setting session status and incrementing frame counts, and can broadcast messages to all connected users.

src/model/object_model.py	Contains the ObjectDetector class. At initialisation, it uses PyTorch to detect whether a CUDA-capable GPU is available and selects the appropriate device. It then loads the specified YOLOv8 model weights. The detect() method accepts a BGR image array from OpenCV, runs YOLOv8 inference with optional class filtering, and if annotate=True, draws colour-coded bounding boxes directly on the frame. A deterministic colour function assigns a consistent colour to each class name.
src/backend/auth.py	Handles the entire API key lifecycle. It creates and manages the SQLite database at data/keys.db, provides functions to generate new keys (using Python's secrets module for cryptographic randomness), hash them with SHA-256 before storage, verify incoming keys by hashing and comparing, list all active keys with their metadata, and revoke keys by database ID. It exposes FastAPI Depend functions (require_api_key, require_admin, verify_ws_key) that are injected into endpoint definitions.
src/backend/rate_limiter.py	Implements rate limiting at three granularities: standard API requests, file uploads, and WebSocket connections. Each limit bucket is tracked per API key. When a limit is exceeded, the module raises an HTTPException with status 429, which FastAPI automatically converts into a proper HTTP error response. Rate limit headers are included in successful responses so clients know how many requests they have remaining.
src/backend/logger.py	Provides two logging capabilities. The RequestLoggingMiddleware is a Starlette middleware class that intercepts every HTTP request and response, recording details such as the HTTP method, URL path, response status code, and response time to logs/access.log. The log_inference() function records the result of each completed WebSocket inference session to logs/inference.log. Both files use JSON-per-line format, which means each log entry is a valid JSON object on its own line — this format is widely used in production systems and can be easily ingested by log aggregation tools.
src/frontend/index.html	A complete, self-contained single-page application in a single HTML file. It includes embedded CSS for the dark-theme visual design, and embedded JavaScript that manages the WebSocket connection, sends the configuration and video bytes, receives the frame responses, renders each frame to an HTML canvas element, updates the progress bar, and appends detection results to the log panel. No external JavaScript frameworks are required — the entire UI is built with vanilla browser APIs.

2.4 Project File Structure

Below is the complete directory layout with annotations explaining every file and folder. Understanding this structure is the first step when you open the project for the first time.

```
Real-Time-Object-Detector/
│
├── app.py              ← START HERE. Run this to launch the server.
├── requirements.txt        ← All Python library dependencies.
├── setup.sh            ← One-command installer (creates venv + downloads model).
├── .env                ← Configuration file. Edit this before running.
```

```
├── .gitignore              ← Tells Git which files to exclude from version control.
│
├── src/                    ← All application source code lives here.
│   │
│   ├── server.py           ← FastAPI app: REST endpoints + WebSocket handler.
│   ├── websocket_handler.py  ← Manages up to 50 concurrent WebSocket sessions.
│   ├── __init__.py         ← Makes 'src' a Python package.
│   │
│   ├── frontend/
│   │   └── index.html      ← The complete browser UI (served at /static/).
│   │
│   ├── backend/            ← Security and infrastructure utilities.
│   │   ├── auth.py         ← API key generation, hashing, and verification.
│   │   ├── logger.py       ← HTTP request logging + inference session logging.
│   │   ├── rate_limiter.py ← Per-key request rate limiting.
│   │   └── __init__.py     ← Exports middleware and dependency functions.
│   │
│   └── model/
│       ├── object_model.py ← YOLOv8 wrapper: load, detect, annotate frames.
│       └── __init__.py
│
├── data/
│   └── keys.db             ← SQLite database storing hashed API keys.
│
├── logs/
│   ├── access.log          ← Every HTTP request (JSON per line).
│   └── inference.log       ← Every completed inference session (JSON per line).
│
└── Video_storage/          ← Uploaded videos, organised by date.
    └── YYYY-MM-DD/
        └── filename_a1b2c3d4.mp4
```

Chapter 3 — Installation and Quick Start Guide

This chapter walks you through setting up and running ObjectVision on your own machine. The instructions are written for students with a basic familiarity with the command line. Every step is explained in detail so you understand not just what to do, but why you are doing it.

3.1 Prerequisites

Before installing ObjectVision, make sure your system meets the following requirements:

- **Python 3.10 or Higher:** ObjectVision uses modern Python features including the match statement (structural pattern matching) and improved type hints. Open a terminal and run 'python --version' to check. If you have an older version, download Python from the official website at python.org.
- **pip (Python Package Installer):** pip comes bundled with Python 3.4 and above. It is used by the setup script to install all required libraries. Run 'pip --version' to confirm it is available.
- **Git (Optional but Recommended):** If you are cloning the project from a repository, you will need Git. Run 'git --version' to check. Alternatively, you can download the project as a ZIP file.
- **For GPU Acceleration (Optional):** If your machine has an NVIDIA graphics card with CUDA support, ObjectVision can use it to run inference significantly faster. You will need the NVIDIA driver version 525 or higher and CUDA Toolkit version 12.x installed. Run 'nvidia-smi' in your terminal to check your driver version. If you see output with your GPU details, GPU acceleration is available. If not, ObjectVision will still run correctly on CPU — just more slowly.
- **Available Disk Space:** The YOLOv8 Nano model (the default) is approximately 6 MB. The largest model (World) is approximately 200 MB. Uploaded videos are also stored on disk, so ensure you have at least 1 GB of free space for comfortable operation.

> **Note for Workshop Participants:**
> The workshop environment (Google Colab / provided cloud VM) already has all prerequisites installed. If you are using the workshop-provided environment, you can skip directly to Section 3.2 Step 2.

3.2 Installation Steps

Step 1 — Obtain the Project Files

If you have the ZIP file provided during the workshop, unzip it to a folder of your choice. If you are working from a Git repository, open a terminal, navigate to the folder where you want to store the project, and clone the repository.

Once you have the files, open a terminal window and navigate into the project folder:

```
cd Real-Time-Object-Detector
```

Step 2 — Run the Setup Script

The project includes a shell script called setup.sh that automates the entire installation. It creates a Python virtual environment (an isolated container for Python packages that does not interfere with the rest of your system), installs all required libraries, and triggers the initial download of the default YOLOv8 model weights.

For a machine without a GPU (most laptops):

```
bash setup.sh
```

For a machine with an NVIDIA CUDA-capable GPU:

```
bash setup.sh --gpu
```

> **What is a Virtual Environment?**
> A virtual environment is an isolated Python installation that lives inside your project folder, inside a directory usually named .venv. When you install packages into a virtual environment, they do not affect the global Python installation on your machine, and they do not conflict with packages from other projects. This is considered best practice in Python development.

The setup script may take several minutes to complete, as it needs to download and install libraries like PyTorch, Ultralytics, FastAPI, and OpenCV. You will see progress messages in the terminal.

Step 3 — Configure Your Environment

All of ObjectVision's runtime settings are controlled by a file named .env in the project root. This file contains key-value pairs that configure the server address, port, which AI model to load, authentication settings, and more.

Open the .env file in a text editor. For local development and workshop exercises, you can use the default settings as-is. The most important setting to be aware of is AUTH_ENABLED:

```
# Set to false to disable API key checks during development
AUTH_ENABLED=false
```

When AUTH_ENABLED is set to false, the server will accept all requests without requiring an API key. This is convenient for testing but should never be used in a public-facing deployment.

> **Security Warning:**
> Never share your .env file publicly or commit it to a Git repository. It may contain your ADMIN_KEY and other sensitive configuration values. The project's .gitignore file is already configured to exclude .env from version control.

Step 4 — Activate the Virtual Environment and Start the Server

Before running the application, you must activate the virtual environment. This tells your terminal to use the Python and packages installed by setup.sh rather than the global Python installation.

On Linux and macOS:

```
source .venv/bin/activate
```

On Windows (Command Prompt):

```
.venv\Scripts\activate.bat
```

On Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Once activated, your terminal prompt will show the name of the virtual environment (typically '(.venv)') at the beginning of each line. Now start the server:

```
python app.py
```

You will see a startup banner in the terminal confirming the server is running and listing the URLs you can visit.

Step 5 — Open the Application

With the server running, open a web browser and navigate to one of the following addresses:

- **Main User Interface:** http://localhost:8000/static/index.html
- **Swagger Interactive API Documentation:** http://localhost:8000/docs
- **Health Check (JSON response):** http://localhost:8000/

The Swagger documentation at /docs is particularly useful for students: it provides an interactive interface where you can try out every API endpoint directly from the browser, see the expected request format, and examine the response structure — all without writing a single line of code.

Chapter 4 — API Reference

An API (Application Programming Interface) is a defined set of rules for how different pieces of software communicate with each other. ObjectVision exposes its functionality through an HTTP REST API and a WebSocket API. Understanding this API is essential for anyone who wants to integrate ObjectVision into their own application, automate requests using a script, or simply understand how the browser UI works under the hood.

In a REST API, each 'endpoint' is a URL path that accepts a specific type of HTTP request (GET to retrieve data, POST to send data, DELETE to remove something). The server reads the request, performs the appropriate action, and returns a response, typically in JSON format.

Authentication: Unless AUTH_ENABLED is set to false, all protected endpoints require you to include your API key in the request header as X-API-Key: ov_your_key_here. For the WebSocket endpoint, the key is passed as a URL query parameter: /ws/inference?api_key=ov_your_key_here.

4.1 Complete Endpoint Reference

Method	Path	Description	Auth Level
GET	/	Health check. Returns the service name, current active model, hardware device (CPU or CUDA), and whether authentication is enabled. Use this to confirm the server is running correctly.	*Public*
GET	/static/index.html	Serves the complete browser-based user interface. This is the page you open when using ObjectVision normally.	*Public*
GET	/api/models	Returns a list of all six supported YOLOv8 model variants, including their parameter counts and mAP50-95 accuracy scores. Also returns the name of the currently loaded model.	*API Key*
POST	/api/upload	Accepts a video file upload. The file is validated (must be a video content type, must not exceed the configured size limit), saved to Video_storage/YYYY-MM-DD/ with a unique suffix, and the saved path is returned.	*API Key*
GET	/api/storage	Returns a structured listing of all stored videos, grouped by the date they were uploaded. Each entry includes the filename and file size in megabytes.	*API Key*
WS	/ws/inference	The main WebSocket endpoint for real-time object detection. Accepts a JSON config frame followed by binary video bytes, then streams annotated frames back to the client. Full protocol described in Chapter 5.	*?api_key=*
GET	/ws/status	Returns a summary of all active WebSocket sessions: how many are currently connected, the	*API Key*

		server capacity, and the status of each session (connecting, receiving, running, done).	
GET	/admin/keys	Administrator endpoint. Returns all API keys in the database with their label, the first ten characters of the key (for identification), creation date, last used date, and total use count. The full plaintext key is never shown.	*Admin Key*
POST	/admin/keys	Administrator endpoint. Creates a new API key with an optional descriptive label. The response includes the full plaintext key exactly once. After this response is received, the key cannot be recovered — only the hash is stored.	*Admin Key*
DELETE	/admin/keys/{id}	Administrator endpoint. Revokes the API key with the specified database ID. The key is marked inactive immediately — any in-flight request using that key will fail.	*Admin Key*
GET	/admin/logs	Administrator endpoint. Returns the last N lines from either the access log or the inference log. Useful for monitoring server activity and debugging issues without needing SSH access to the server.	*Admin Key*

4.2 Understanding HTTP Methods

If you are new to web APIs, the HTTP method (GET, POST, DELETE) is a crucial concept to understand. It describes the intent of the request:

- **GET:** Retrieve information without changing anything. Safe to call repeatedly. Example: checking the health of the server or listing stored videos.
- **POST:** Send data to the server to create or trigger something. Example: uploading a video file or creating a new API key.
- **DELETE:** Request the removal of a resource. Example: revoking an API key by its ID.
- **WS (WebSocket):** Establish a persistent bidirectional connection for real-time communication. Different from REST — once connected, both the client and server can send messages at any time.

4.3 Rate Limiting

Rate limiting is a technique used to control how frequently a client can make requests to a server. Without rate limiting, a single user could flood the server with requests, consuming all available resources and preventing other users from getting a response. ObjectVision implements three separate rate limit buckets:

- **Standard API Requests (GET endpoints):** A general-purpose limit that applies to all read operations like listing models, checking storage, and viewing WebSocket status.
- **File Uploads (POST /api/upload):** A stricter limit, because each upload consumes disk space and processing time. This prevents a single user from filling up the server's storage.
- **WebSocket Connections (/ws/inference):** Checked once at connection time. This prevents a single key from opening dozens of simultaneous inference sessions.

When a rate limit is exceeded, the server responds with HTTP status 429 (Too Many Requests) for REST endpoints, or closes the WebSocket with code 4029 for connections. Rate limit information is included in the response headers so clients can implement intelligent back-off strategies.

Chapter 5 — The WebSocket Inference Protocol

The WebSocket protocol is central to ObjectVision's real-time capability. This chapter explains what a WebSocket is, why it is used here, and documents the exact message format and sequence that the client and server follow during an inference session. Students who want to build their own client application or integrate ObjectVision into another project must read this chapter carefully.

5.1 What is a WebSocket and Why Use It?

To understand WebSockets, it helps to first understand standard HTTP. In a typical HTTP interaction, the client (your browser) sends a request to the server, the server processes it and sends back a response, and then the connection is closed. This request-response pattern works perfectly for loading web pages and fetching data, but it is not suitable for real-time streaming.

Imagine if ObjectVision used standard HTTP to deliver annotated frames. After processing each frame, the server would need to wait for the browser to ask for it. The browser would need to send hundreds of requests per second. Each request would have overhead from headers and connection setup. The result would be slow, choppy, and resource-intensive.

A WebSocket solves this problem entirely. The connection is established once with a single HTTP 'upgrade' request, and then it stays open as a persistent, full-duplex channel. This means the server can push frames to the browser the moment they are ready, without any waiting. The browser does not need to ask — it simply receives. This is why real-time video streaming feels smooth in ObjectVision.

5.2 Connection Lifecycle and Message Sequence

Every ObjectVision inference session follows an exact four-phase sequence. Deviating from this sequence will cause the server to reject the connection or fail to process the video correctly.

Phase 1 — Authentication and Connection

The browser appends the API key as a query parameter to the WebSocket URL and initiates the connection:

```
ws://localhost:8000/ws/inference?api_key=ov_a1b2c3d4e5f6...
```

Before accepting the connection, the server validates the key. If the key is missing or invalid, the server accepts the WebSocket (this is required by the WebSocket protocol before any messages can be exchanged), immediately sends an error JSON message, and then closes the connection with code 4001. This means the browser momentarily opens a connection only to have it closed — clients must handle this case.

If the key is valid, the rate limiter checks whether this key has remaining connection budget. If the limit is exceeded, the connection is closed with code 4029.

If both checks pass, the ConnectionManager is called. If the server is already handling 50 simultaneous sessions, the new connection is rejected with code 1008 (Policy Violation). Otherwise, the session is registered and the connection is fully established.

Phase 2 — Configuration Frame

The browser immediately sends a text WebSocket frame containing a JSON object that configures the inference session. This must be the very first message after the connection is established:

```
{
 "type":       "config",
 "session_id": "SID-XXXXX",      ← Any identifier string for this session
 "confidence": 0.5,              ← Minimum detection confidence (0.0 to 1.0)
 "classes":    ["person", "car"], ← Only detect these classes (empty = all 80)
```

```
  "filename":   "my_video.mp4",    ← The original filename of the video
  "model":      "yolov8n.pt"       ← Which model variant to use
}
```

The confidence threshold is particularly important to understand. A value of 0.5 means: only include a detection in the results if the model is at least 50% confident that the identified object is really there. Setting it too low (e.g., 0.1) will produce many false positives — the model will 'see' objects that aren't there. Setting it too high (e.g., 0.9) will cause the model to miss real objects it is not fully certain about. For most applications, a value between 0.4 and 0.6 is a good starting point.

Phase 3 — Binary Video Upload

Immediately after sending the config frame, the browser sends a binary WebSocket frame containing the raw bytes of the video file. There is no compression or special encoding — just the file bytes exactly as they would appear on disk.

The server receives these bytes, writes them to the dated storage directory with a unique filename suffix, and sends back a status message confirming the save path. The model then loads the video using OpenCV's VideoCapture and begins processing frames.

Phase 4 — Streaming Frame Results

For every frame in the video, the server sends a JSON text frame back to the browser. The browser does not need to request frames — they arrive as fast as the server can process them, up to the configured maximum frame rate. Each message has the following structure:

```
{
  "type":      "frame",
  "frame":     42,              ← Sequential frame index (starts at 1)
  "fps":       28.3,            ← Live frames-per-second being processed
  "progress":  35.0,            ← Percentage of video completed
  "image":     "<base64 JPEG>",  ← The annotated frame, base64-encoded
  "detections": [               ← List of objects found in this frame
    { "cls": "person", "conf": 0.923 },
    { "cls": "car",    "conf": 0.871 }
  ]
}
```

The image field contains the annotated video frame encoded as a JPEG image (at 75% quality to balance file size and visual quality) and then converted to a Base64 string. Base64 is an encoding scheme that converts binary data (like image bytes) into a string of printable ASCII characters, making it safe to transmit inside a JSON text message. The browser decodes this string back to an image and draws it on the canvas element.

5.3 Server Status and Error Messages

The server sends several other types of messages throughout the session lifecycle:

Type Field	When Sent	What It Contains and What It Means
status	During setup	A human-readable message field describing the current state. Examples: 'Loading yolov8m.pt...' (if a new model needs to be loaded), 'yolov8m.pt ready' (model loaded successfully), 'Saved → Video_storage/2026-03-25/myvideo_a1b2c3d4.mp4' (video saved to disk).

frame	Per video frame	The main data message, sent once per video frame. Contains the annotated image and detection list as described in Section 5.2 above.
done	End of video	Signals that all frames have been processed. Contains total_frames (the total number of frames in the video) and saved_path (the full path to the stored video file). After receiving this message, the client should close the WebSocket connection.
error	On any failure	Contains a message field with a human-readable error description. Sent when authentication fails, when the server is at capacity, when the video file cannot be opened, or when any unexpected exception occurs during processing. After an error message, the connection is typically closed.

Chapter 6 — YOLOv8 Model Reference

This chapter explains what YOLOv8 is, how the different model variants differ from one another, and how to choose the right model for your use case. Understanding this will help you make informed decisions when experimenting with the system during the workshop.

6.1 What is YOLO?

YOLO stands for 'You Only Look Once'. It is a family of deep learning models designed specifically for object detection. The name comes from a fundamental architectural choice that distinguishes it from earlier detection approaches: YOLO processes the entire image in a single pass through the neural network, rather than scanning the image in multiple stages.

Older object detection systems typically worked in two stages: first, they would propose regions of the image that might contain objects (the 'region proposal' stage), and then they would classify each proposed region separately. This two-stage approach was accurate but slow, often taking seconds per image. YOLO unified these stages into one, making real-time detection possible.

YOLOv8, released by Ultralytics, is the eighth major iteration of the YOLO architecture. It introduces improvements in accuracy, speed, and ease of use compared to its predecessors. It also provides a clean Python API, automatic model downloading, and support for multiple tasks beyond object detection, including image segmentation and pose estimation.

6.2 Model Variants and the Speed-Accuracy Trade-off

YOLOv8 comes in six variants, named after their relative size: Nano, Small, Medium, Large, XLarge, and World. Each variant uses a different neural network architecture with a different number of parameters. More parameters generally means better accuracy (the model has more capacity to learn complex patterns), but also means the model is slower to run and requires more memory.

The accuracy metric used in the table below is mAP50-95, which stands for Mean Average Precision calculated at Intersection over Union (IoU) thresholds ranging from 0.50 to 0.95. Without going deep into the mathematics, a higher mAP50-95 value means the model is better at both correctly identifying the class of detected objects and precisely drawing the bounding box around them. The scores are benchmarked on the COCO val2017 dataset, which is the standard evaluation set used by the entire computer vision research community.

Model File	Label	Variant	Parameters	mAP50-95	Relative Performance Bar
yolov8n.pt ← Default	**v8n**	Nano	3.2M	**37.3**	
yolov8s.pt	**v8s**	Small	11.2M	**44.9**	
yolov8m.pt	**v8m**	Medium	25.9M	**50.2**	
yolov8l.pt	**v8l**	Large	43.7M	**52.9**	
yolov8x.pt	**v8x**	XLarge	68.2M	**53.9**	

yolov8x6.pt	**v8x6**	World	97.2M	**56.7**	

> **Which model should I use?**
> For workshop exercises and development on a laptop CPU, start with Nano (yolov8n.pt). It loads in seconds and runs quickly enough to see real-time results. When you want to explore the difference accuracy makes, switch to Small or Medium. If you have access to a GPU (like the workshop L4 cloud instance), try Large or XLarge for noticeably sharper detection results.

6.3 Hot-Swapping Models at Runtime

One of the most useful features of ObjectVision is the ability to change the active model without restarting the server. When you send a WebSocket config frame with a different model name, the server detects the change, loads the new weights, and confirms with a status message before processing begins. This allows you to compare model performance on the same video without any downtime.

Technically, the ObjectDetector class in src/model/object_model.py implements this through its load_model() method. When called with a new model path, it replaces the self.model reference with a fresh YOLO instance and rebuilds the internal class-name-to-ID mapping dictionary. Since the ObjectDetector is a singleton (one instance shared across all WebSocket sessions), model changes affect all future sessions immediately.

> **Important for Students:**
> If two users are simultaneously running inference and one of them triggers a model swap, the other user's session will be unaffected — they have already started streaming frames with the previously loaded model. The new model only applies to sessions that begin after the swap.

6.4 The COCO 80 Object Classes

All YOLOv8 models included in ObjectVision are trained on the COCO (Common Objects in Context) dataset, which defines 80 categories of everyday objects. Class filtering in ObjectVision works by specifying the names of the classes you want to detect in the WebSocket config frame. The server maps these names to their numeric IDs internally and passes them to YOLOv8's predict() method, which filters at the model level for efficiency.

The 80 COCO classes span the following categories:

- **People:** person
- **Vehicles:** bicycle, car, motorcycle, airplane, bus, train, truck, boat
- **Traffic:** traffic light, fire hydrant, stop sign, parking meter
- **Outdoor:** bench
- **Animals:** bird, cat, dog, horse, sheep, cow, elephant, bear, zebra, giraffe
- **Accessories:** backpack, umbrella, handbag, tie, suitcase
- **Sports:** frisbee, skis, snowboard, sports ball, kite, baseball bat, baseball glove, skateboard, surfboard, tennis racket
- **Kitchen:** bottle, wine glass, cup, fork, knife, spoon, bowl
- **Food:** banana, apple, sandwich, orange, broccoli, carrot, hot dog, pizza, donut, cake
- **Furniture:** chair, couch, potted plant, bed, dining table, toilet
- **Electronics:** tv, laptop, mouse, remote, keyboard, cell phone, microwave, oven, toaster

- **Appliances:** sink, refrigerator
- **Indoor Objects:** book, clock, vase, scissors, teddy bear, hair drier, toothbrush

Chapter 7 — Authentication and API Key Management

Authentication is the process of verifying the identity of a user or client application before granting access to a protected resource. In ObjectVision, authentication is implemented using API keys — long, random strings that act as passwords for programmatic access. This chapter explains how the authentication system works, how keys are managed, and the cryptographic principles behind secure key storage.

7.1 Understanding API Keys

An API key is a randomly generated string of characters that is given to a user or application to identify itself when making requests to an API. Think of it like a combination of a username and password packed into a single string. Unlike a username-password pair where you can choose something memorable, an API key is generated by the server using a cryptographically secure random number generator, making it practically impossible to guess.

ObjectVision keys follow the format ov_<32 random hexadecimal characters>. For example: ov_a1b2c3d4e5f6789012345678abcdef01. The 'ov_' prefix makes it immediately recognisable as an ObjectVision key if you encounter it in a log file or configuration. The 32 hex characters provide 128 bits of entropy — meaning there are 2^128 (approximately 340 undecillion) possible keys. The probability of guessing a valid key by chance is, for all practical purposes, zero.

7.2 Secure Storage with SHA-256 Hashing

When you generate a key, ObjectVision stores it in the SQLite database as a SHA-256 hash, not as the plaintext string. This is a critical security practice. SHA-256 is a one-way cryptographic hash function: given a key, you can always compute its hash, but given only the hash, you cannot recover the original key. This means that even if an attacker gains access to the keys.db database file, they obtain only hashes — which are useless for making API requests.

When a client makes a request with their API key, the server hashes the provided key and compares it to the stored hashes. If a match is found, the request is authenticated. The plaintext key is never stored anywhere and is shown to the user exactly once — at the moment of creation. After that, if the key is lost, it cannot be recovered and a new one must be generated.

> **Cryptographic Note for Students:**
> SHA-256 is part of the SHA-2 (Secure Hash Algorithm 2) family, standardised by the US National Institute of Standards and Technology (NIST). It produces a 256-bit (32-byte) output regardless of input length. Modern applications typically also add a 'salt' (a random value added to the input before hashing) to protect against rainbow table attacks. ObjectVision uses Python's hashlib.sha256() function from the standard library.

7.3 Key Management via Command Line

The auth module can be used as a command-line tool for managing keys directly from the terminal. This is useful for setting up the system before deployment or for administrative tasks.

Generating a new API key with a descriptive label:

```
python -m src.backend.auth keygen --label "workshop-student-01"
```

The terminal will display the new key exactly once. Copy it immediately and store it securely — it cannot be shown again.

Listing all keys currently in the database:

```
python -m src.backend.auth list
```

This shows each key's database ID, label, first ten characters (for identification), creation date, last used date, and how many times it has been used. The full key is never shown.

Revoking a key by its database ID:

```
python -m src.backend.auth revoke 3
```

Replace 3 with the actual database ID shown in the list output. The key is marked inactive immediately.

7.4 Key Management via the Admin API

The same operations are available through the REST API, protected by the ADMIN_KEY configured in .env. This allows remote administration without needing terminal access to the server machine.

Method	Endpoint	What It Does
POST	POST /admin/keys	Creates a new API key. Include a JSON body with an optional 'label' field to give the key a descriptive name. The response includes the full plaintext key under the 'key' field, along with a warning reminding you to save it immediately.
GET	GET /admin/keys	Lists all keys with their metadata. Returns an array of objects, each containing id, label, key_prefix (first ten chars), created_at, last_used, use_count, and is_active.
DELETE	DELETE /admin/keys/{id}	Revokes the key with the specified numeric ID. Returns a confirmation object with the status 'revoked' and the key_id.

7.5 Disabling Authentication for Development

During development and workshop exercises, authentication can be completely disabled by setting AUTH_ENABLED=false in the .env file. When this flag is set, the require_api_key and verify_ws_key functions return immediately without performing any checks, and all endpoints become publicly accessible.

> **Best Practice for Students:**
> Always develop with AUTH_ENABLED=false for convenience, but before deploying anything to a public server or cloud environment, set AUTH_ENABLED=true, generate a strong ADMIN_KEY, and create individual API keys for each user. This is the minimal security baseline for any production deployment.

Chapter 8 — Environment Configuration Reference

All runtime behaviour of ObjectVision is controlled by variables defined in the .env file located in the project root. This design pattern — called 'configuration via environment' — is a best practice in modern software development known as the Twelve-Factor App methodology. It means you can change how the application behaves without editing any source code, and you can run the same codebase with different configurations in development, testing, and production environments.

The .env file is loaded at startup by the python-dotenv library, which reads each line of the form KEY=VALUE and makes these values available via Python's os.getenv() function. If a variable is not set in .env, each os.getenv() call in the code specifies a sensible default value.

Variable Name	Default Value	Detailed Description
HOST	0.0.0.0	The network interface the server binds to. '0.0.0.0' means the server accepts connections from any network interface — this includes your local machine as well as other devices on the same network. To restrict access to your local machine only, change this to '127.0.0.1'.
PORT	8000	The TCP port number the server listens on. Port 8000 is conventional for development web servers. If you get an error saying the port is already in use, change this to another number such as 8080 or 8001.
RELOAD	true	When set to true, Uvicorn watches your source files for changes and automatically restarts the server when it detects a modification. This is extremely convenient during development. Set to false in production, as the file-watching process adds overhead.
WORKERS	1	The number of Uvicorn worker processes to run simultaneously. Multiple workers allow the server to handle more concurrent requests. This setting has no effect when RELOAD=true, because hot-reload mode only supports a single worker process.
LOG_LEVEL	info	Controls how much information Uvicorn prints to the terminal. Options from most to least verbose: debug (prints every detail including individual request headers), info (standard operational messages), warning (only potential issues), error (only errors that require attention). Use 'debug' when troubleshooting.
YOLO_MODEL	yolov8n.pt	The name of the YOLOv8 model weights file to load at server startup. Must be one of the six supported variants. If the file does not exist locally, Ultralytics will automatically download it from the internet the first time the server starts. The Nano model is the default as it is the smallest and fastest.
MAX_FILE_MB	500	The maximum allowed size in megabytes for a video uploaded via POST /api/upload. If the uploaded file exceeds this limit, the server responds with HTTP status

		413 (Request Entity Too Large) without saving the file. Increase this if you work with high-definition videos.
MAX_STREAM_FPS	30	The maximum number of frames per second the server will stream to a WebSocket client. This acts as a throttle: if the model can process frames faster than this limit, the server inserts a small delay between frames. This prevents fast CPUs or GPUs from overwhelming slow browser connections.
MAX_WS_CONNECTIONS	50	The maximum number of simultaneous WebSocket inference sessions the server will accept. When this limit is reached, new connections are refused with WebSocket close code 1008 (Policy Violation) and an error message. Adjust based on your server's available memory and CPU/GPU capacity.
AUTH_ENABLED	false	Master switch for the API key authentication system. Set to 'true' to require a valid API key for all protected endpoints. Set to 'false' to bypass all authentication checks — convenient for local development but never appropriate for a publicly accessible deployment.
AUTH_DB_PATH	data/keys.db	The file system path to the SQLite database file that stores hashed API keys. The directory is created automatically if it does not exist. You can change this path if you want to store the database in a different location, such as a mounted volume in a Docker deployment.
ADMIN_KEY	(empty)	If set, this value acts as a master administrator password for the /admin/* endpoints. Only requests that include this exact value as the X-Admin-Key header will be granted access to key management and log viewing. Choose a long, random string. If left empty, admin endpoints may behave differently depending on AUTH_ENABLED.
VIDEO_STORAGE_ROOT	Video_storage	The root directory where uploaded videos are saved. Inside this directory, the server automatically creates sub-directories named by date (YYYY-MM-DD format). Each uploaded video is stored in today's sub-directory. You can change this to an absolute path to store videos on a different disk or volume.
LOG_DIR	logs	The directory where the access.log and inference.log files are created. Both files use JSON-per-line format (also called NDJSON or newline-delimited JSON), where each line is a complete, valid JSON object representing one log event. This format is widely supported by log analysis tools.

Chapter 9 — Video Storage and Logging

This chapter covers two important operational aspects of ObjectVision: how uploaded videos are organised on disk, and how the system records its activity through structured log files. Understanding these aspects is important for students who want to deploy ObjectVision in a real environment, debug issues, or analyse usage patterns.

9.1 Video Storage Organisation

Every video that passes through ObjectVision — whether uploaded via the REST endpoint or received through the WebSocket inference stream — is automatically saved to the file system in an organised structure. The root storage directory (configurable via VIDEO_STORAGE_ROOT in .env, defaulting to a folder named Video_storage) contains date-named sub-directories. Each day's sub-directory holds all videos that were processed on that date.

This date-based organisation has several practical advantages: it is easy to find videos from a particular day, it prevents the root directory from becoming cluttered with thousands of files, and it simplifies archival — you can easily delete or archive older sub-directories without touching recent uploads.

```
Video_storage/
├── 2026-03-25/
│   ├── workshop_demo_a1b2c3d4.mp4     ← Original name + UUID suffix
│   └── test_video_e5f6g7h8.avi
├── 2026-03-26/
│   └── campus_footage_i9j0k1l2.mp4
└── 2026-03-27/
    └── traffic_sample_m3n4o5p6.webm
```

9.2 File Naming Convention

Each saved video file follows a specific naming pattern: the original filename stem (the name without the extension) is retained, then an underscore and an eight-character hexadecimal suffix derived from a UUID (Universally Unique Identifier) are appended, followed by the original file extension.

For example, if you upload a file named workshop_demo.mp4, it might be saved as workshop_demo_a1b2c3d4.mp4. The UUID suffix serves an important purpose: it ensures that two files with the same original name, uploaded on the same day, will not overwrite each other. Without this suffix, the second upload would silently replace the first.

The UUID is generated using Python's uuid.uuid4().hex[:8], which creates a random Version 4 UUID, converts it to a hexadecimal string, and takes the first eight characters. While there is a theoretical probability of two uploads generating the same eight-character suffix on the same day, the probability is so low (approximately 1 in 4 billion) that it is negligible in practice.

9.3 Supported Video Formats

ObjectVision accepts and correctly processes video files in the following container formats:

Extension	Format Name	Notes
.mp4	MPEG-4 Part 14	The most widely used video format. Recommended for best compatibility. Most cameras, smartphones, and screen recording tools produce MP4 files.

.avi	Audio Video Interleave	An older Microsoft format. Still widely used in research and legacy systems. OpenCV has excellent support for AVI files.
.mov	QuickTime Movie	Apple's native video format. Common output from iPhones and macOS screen recorders. Generally well supported by OpenCV.
.mkv	Matroska Video	An open-source container format that can hold almost any codec. Popular for high-quality video storage and distribution.
.webm	WebM	An open, web-optimised format developed by Google. Common output from web-based screen recorders and video platforms.

9.4 Understanding the Log Files

ObjectVision maintains two log files in the logs/ directory. Both use JSON-per-line format, meaning each log entry is a complete JSON object on its own line. This format is machine-readable, human-readable, and compatible with popular log analysis tools such as jq, Logstash, Splunk, and Grafana Loki.

The Access Log (logs/access.log)

Every HTTP request received by the server is recorded here by the RequestLoggingMiddleware. Each entry includes the HTTP method, the full URL path, the response status code, and the time taken to process the request in milliseconds. This log is useful for understanding usage patterns, identifying slow endpoints, and detecting unusual request volumes that might indicate abuse.

Example access log entry:

```
{ "method": "POST", "path": "/api/upload", "status": 200, "duration_ms": 142 }
```

The Inference Log (logs/inference.log)

Every completed WebSocket inference session is recorded here by the log_inference() function called at the end of each session. Each entry is a rich summary of the session including:

- **session_id:** The unique identifier assigned to this session.
- **model:** The name of the YOLOv8 model variant that was used.
- **confidence:** The confidence threshold that was configured for this session.
- **classes_count:** How many object classes were included in the filter (zero means all 80 classes were active).
- **total_frames:** The total number of frames processed from the video.
- **duration_s:** The total time in seconds from connection to completion.
- **detections_total:** The cumulative number of object detections across all frames.
- **ip:** The IP address of the client who ran this session.
- **api_key_id:** The first ten characters of the API key used (for identification without revealing the full key).
- **saved_path:** The full file system path where the uploaded video was saved.

This data enables meaningful analysis: which models are most popular, what is the average session duration, how many detections per frame does each video produce, and which users are most active.

Viewing Logs via the Admin API

Rather than requiring SSH access to view log files, ObjectVision provides a dedicated admin endpoint for tailing logs remotely. The GET /admin/logs endpoint accepts two optional query parameters:

- **log_type:** Either 'access' or 'inference'. Selects which log file to tail. Defaults to 'access'.
- **lines:** The number of lines to return from the end of the file. Must be between 1 and 500. Defaults to 50.

The endpoint parses each log line as JSON and returns a structured array, so clients can process the log data programmatically without any string parsing.

Chapter 10 — Attribution and Acknowledgements

ObjectVision was conceived, designed, and developed under the auspices of the Gudsky Research Foundation as part of its commitment to making applied artificial intelligence education freely accessible to students and researchers across India and beyond.

10.1 Gudsky Research Foundation

The Gudsky Research Foundation is a Section 8 Non-Profit Organisation registered under Indian law and dedicated to advancing research and education in the fields of Artificial Intelligence, Data Science, and Emerging Technologies. Section 8 of the Companies Act, 2013 governs organisations formed for the promotion of science, art, education, research, or any other useful object, where profits are applied for promoting those objectives and no dividend is paid to members.

The Foundation holds the following certifications and registrations, each of which reflects its commitment to standards, transparency, and institutional accountability:

- **ISO Certified:** The Foundation's processes and quality management systems conform to international ISO standards, ensuring consistency and reliability in its research and educational outputs.
- **MSME Certified:** Registered under the Ministry of Micro, Small and Medium Enterprises, enabling the Foundation to access government support and participate in schemes designed to foster innovation.
- **AICTE Approved:** The All India Council for Technical Education has approved the Foundation's technical education initiatives, lending them national recognition and academic credibility.
- **Startup India — DPIIT Registered:** Registered under the Department for Promotion of Industry and Internal Trade's Startup India programme, recognising the Foundation's innovative approach to education technology.
- **NGO Darpan Registered:** Listed on the NGO Darpan portal of NITI Aayog, the Government of India's policy think tank, ensuring full transparency and accountability in the Foundation's operations.

10.2 Virtual Research Center

ObjectVision was developed at the Virtual Research Center (VRC), the Foundation's dedicated environment for applied AI research and prototype development. The VRC provides students and researchers with cloud infrastructure, development tools, datasets, and mentorship to build real-world AI applications. Projects developed at the VRC are intended to bridge the gap between academic knowledge and industry practice.

10.3 Dronacharya Free Learning Initiative

The Dronacharya Free Learning initiative is the Foundation's flagship educational programme, named after Dronacharya, the revered teacher of the Mahabharata who is considered the ideal of the guru-shishya (teacher-student) tradition in Indian culture. The initiative is built on the principle that high-quality technical education should be accessible to every student regardless of economic background.

ObjectVision was created as a workshop project under this initiative and is shared openly with students as a learning resource. Students are encouraged to study the code, experiment with it, break it, fix it, and build upon it.

10.4 The Workshop

This documentation accompanies the project delivered during the following workshop:

Detail	Information
Workshop Title	AI Model Creation and Deployment
Format	5-Day Hands-On Workshop
Dates	25 March 2026 to 29 March 2026
Delivery Mode	Online, live sessions with real-time interaction
Cost	100% Free of Charge — open to all students
Organiser	Gudsky Research Foundation — Dronacharya Free Learning

ObjectVision — Real-Time Object Detection System

Official Project Documentation · Gudsky Research Foundation · Virtual Research Center

Dronacharya Free Learning · Making AI Education Accessible to All